



Mantis

CCPROG2 Machine Project (AY 2025-2026, Term 2)

1 Project Specifications.....	2
1.1 Overview.....	2
1.2 Mantis.....	3
1.3 Program Flow.....	3
1.3.1 Game Proper.....	3
1.3.2 View Statistics.....	7
1.3.3 Settings.....	7
1.4 Interface.....	7
1.4.1 Custom User Interface Library (Optional).....	7
1.5 Mantis Card Format.....	8
1.6 Bonus Points.....	8
1.7 Implementation Requirements.....	9
2 Documentation.....	9
2.1 Internal Code Documentation.....	9
2.1.1 Program Structure.....	9
2.1.2 Functions and Function Comments.....	10
2.1.3 Structures.....	11
2.2 Test Plan.....	11
3 Deliverables.....	12
3.1 Submission & Demo.....	12
3.2 Deliverables Checklist.....	12
3.3 Milestones.....	12
3.4 MP Starter Files.....	13
4 Collaboration and Academic Honesty.....	13
5 Grading.....	13
Acknowledgments / Disclosure.....	13
Appendix A: Test Plan Format Sample.....	14
Appendix B: Export Test Plan Guide.....	15

Release Date: January 30, 2026

Due Date: March 30, 2026 (M) 0730

Prepared by: Kristine Kalaw

1 Project Specifications

You must create a C program based on the project specifications detailed below. This project serves as a culminating program activity to showcase the programming concepts learned in class. Through the project, students should be able to demonstrate the learning outcomes indicated in the syllabus.

For this project, you are allowed to do this **individually** or to be in a **group of 2 members** only. Should you decide to work in a group, the following mechanics apply:

- **Individual Solution:** Even if it is a group project, each student is still required to create their INITIAL solution for the MP individually without discussing it with any other students. This will help ensure that each student goes through the process of reading, understanding, solving the problem, and testing the solution.
- **Group Solution:** Once both students are done with their solution, they discuss and compare their respective solutions (ONLY within the group) – note that learning with a peer is the objective here – to see a possibly different or better way of solving a problem. They then come up with their group's final solution – which may be the solution of one of the students or an improvement over both solutions. Only the group's final solution, with internal documentation (part of the comment indicating whose code was used or was an improved version of both solutions), will be submitted as part of the final deliverables. It is the group solution that will be checked/assessed/graded. Thus, only 1 final set of deliverables should be uploaded by one of the members on the Canvas submission page. [Groupings will be created in AnimoSpace to facilitate this.]
- **Individual Demo Problem:** As each is expected to have solved the MP requirements individually before coming up with the final submission, both members should know all parts of the final code to allow each to INDIVIDUALLY complete the demo problem within a limited amount of time (to be announced nearer the demo schedule). This demo problem is given only on the day/time of the demo and may be different for each member of the group. Both students should be present during the demo, not just to present their individual demo problem solution, but also to answer questions about their group submission.
- **Grading:** The MP grade will be the same for both students – UNLESS there is a compelling and glaring reason as to why one student should get a different grade from the other – for example, one student cannot answer questions properly OR does not know where or how to modify the code to solve the demo problem (in which case deductions may be applied or a 0 grade may be given – to be determined on a case-to-case basis).

1.1 Overview

You are tasked to create a text-based game for facilitating a game of **Mantis**. Within the program, users can (1) manage the game components; (2) view game statistics; and, of course, (3) play the game proper.



(image source: <https://www.gaminglib.com/products/mantis>)

1.2 Mantis

A colorfully cutthroat card game of RAINBOWS and REVENGE.

In the actual card game Mantis, there are only two rules: Steal or Score. With the simplicity of UNO and the depth of Gin Rummy, Mantis is designed for both kids and adults. But adults should be warned that kids will often beat them at this game. You collect matching sets of cards by stealing from your opponents. You feel good about things, then not so good about things as opponents steal your cards back. Repeat this emotional rollercoaster until somebody wins the game. (source: <https://www.explodingkittens.com/products/mantis>).

For your MP, you are to create a program that helps facilitate a modified version of the game. This entails:

- Keeping track of the players who have played the game, their wins, and their highest score
- Displaying the current state of the game per turn
- Allowing the players to score or steal during their turn
- Showing the game results and statistics
- Allowing players to customize the game configuration

For more information about the rules of the Mantis game, you may [view the instructions manual](#) or [watch this video](#).

1.3 Program Flow

When the program starts, it should display the main menu. The main menu contains **at least** the following options:

- **New Game:** If the user selects this option, the program should proceed to the game proper. See [Section 1.3.1](#) for more details.
- **Top Players:** If the user selects this option, the program should display the game statistics. See [Section 1.3.2](#) for more details.
- **Settings:** If the user selects this option, the program should display options to adjust the game configuration. See [Section 1.3.3](#) for more details
- **Exit:** If the user selects this option, the program should terminate without crashing. Note that this is the only valid way for the program to terminate.

```
Main Menu
[1] New Game
[2] Top Players
[3] Settings
[0] Exit

>> 1
```

Sample interface for the main menu

1.3.1 Game Proper

When a game starts, the following flow should be observed:

1. Determine the number of players.

- A game of Mantis has a minimum of 3 players and a maximum of 6 players.
 - *Note that a 2-player variant exists, but it is outside the project's basic requirements.*
- The players of the game can be new or existing. Existing players are chosen from a list. If it is a new player, ask for their username.
 - The username can be up to 36 characters and must be unique.
 - Note that in the next runs of the program, this "new player" should already be part of the existing player list (i.e., changes to the list of players persist).
 - Assume that the program can handle up to a maximum of 50 players.
- The list of players should be found in **players.txt**. You are free to decide on the file format, but take note of at least the following player data: username, number of games won, and highest score.

```

How many players?: 3
P1: NeonNinja
P2: Pl@yer0
P3: ?

Select Player 3:
[0] <Add new player>
[1] nu11
[2] ZzapZzilla

>> 0

```

```
New player username: Jet2HDay
```

Sample interface for player selection and creation

2. Shuffle the deck and deal Tank cards.

- An unshuffled “deck” is provided in **mantis.txt**. Load the content of this file into your program, and you must use the provided **shuffle()** function to shuffle the deck. See [Section 3.4](#) on how to access these.
- Deal 4 cards “face up” to the Tank Pile of each player. The remaining cards remain “face down” and serve as the Draw Pile.
- In the sample interface below:
 - P1 received 1 red card, 1 orange card, 1 yellow card, and 1 green card
 - P2 received 1 orange card, 2 blue cards, and 1 violet card
 - P3 received 1 red card, 1 yellow card, and 2 indigo cards

```

P1 => [ R:1 | O:1 | Y:1 | G:1 | B:0 | I:0 | V:0 ] // 0
P2 => [ R:0 | O:1 | Y:0 | G:0 | B:2 | I:0 | V:1 ] // 0
P3 => [ R:1 | O:0 | Y:1 | G:0 | B:0 | I:2 | V:0 ] // 0

```

Sample interface after dealing the initial Tank cards

3. Game Start!

- At the start of each player’s turn, they will decide to either **Try to Score** or **Try to Steal**.
 - **Try to Score flow:**
 - Take the top card from the deck and turn it over to inspect its front color.
 - If the current player already has at least 1 card with that color in their Tank, all those color cards (including the newly drawn one) are added to their Score Pile. The score is updated based on the number of points indicated on the cards in their Score Pile.
 - If the current player does not have any cards of that color, then it is added to their Tank.
 - **Try to Steal flow:**
 - Choose another player to attempt to steal from.
 - Take the top card from the deck and turn it over to inspect its front color.
 - If the *other player* already has at least 1 card with that color in their Tank, all those color cards (including the newly drawn one) are transferred to the current player’s Tank Pile.
 - If the *other player* does not have any cards of that color, then it is added to the other player’s Tank.
- After resolving the Try to Score or Try to Steal flow, advance to the next player.
- The first player with 20 points in their Score Pile wins.
 - If the Draw Pile runs out, the player with the most points in their Score Pile wins. If there are players with the same number of points in their Score Pile, then the player with the most Tank cards wins. If those players still have the same number of cards in their Tank, then it is a tie. For the purposes of counting wins, a tie result is counted as a win for the involved players.
 - *Note that a modification for this text-based version of Mantis is to have corresponding points to each card.*

4. Show game results

- The winner of the game is revealed.

Try to Score Sample Flow

```

P1 => [ R:1 | O:1 | Y:1 | G:1 | B:0 | I:0 | V:0 ] // 0
P2 => [ R:0 | O:1 | Y:0 | G:0 | B:2 | I:0 | V:1 ] // 0
P3 => [ R:1 | O:0 | Y:1 | G:0 | B:0 | I:2 | V:0 ] // 0

Top Deck: RGB (93 cards remaining in deck)

Player 1, what would you like to do?
[1] Try to Score
[2] Try to Steal

>> 1

```

Sample interface for the start of Player 1's turn

```

Resolving turn for Player 1...
- Drawn card color reveal: R (2 pt/s)!
- Player 1 has (1) R card/s worth a total of
(2) pts!
- +4 points to Player 1's Score pile!

```

```

Resolving turn for Player 1...
- Drawn card color reveal: B (1 pt/s)!
- Player 1 has no B cards...
- Adding drawn card to Player 1's Tank

```

Sample interface for resolving the Try to Score flow.

On the left (Try to Score Flow A) is when the drawn card color exists in the current player's Tank.

On the right (Try to Score Flow B) is when the drawn card color does not yet exist in the current player's Tank.

```

P1 => [ R:0 | O:1 | Y:1 | G:1 | B:0 | I:0 | V:0 ] // 4
P2 => [ R:0 | O:1 | Y:0 | G:0 | B:2 | I:0 | V:1 ] // 0
P3 => [ R:1 | O:0 | Y:1 | G:0 | B:0 | I:2 | V:0 ] // 0

Top Deck: GIV (92 cards remaining in deck)

Player 2, what would you like to do?
[1] Try to Score
[2] Try to Steal

>>

```

*Sample interface after Try to Score Flow A (notice **Player 1's R and Score**)*

For the score computation, the newly drawn card is worth 2 points. Based on the game state during the turn resolution, Player 1's R pile has a total of 2 points, resulting in a total of +4 points in the Score Pile.

```

P1 => [ R:1 | O:1 | Y:1 | G:1 | B:1 | I:0 | V:0 ] // 0
P2 => [ R:0 | O:1 | Y:0 | G:0 | B:2 | I:0 | V:1 ] // 0
P3 => [ R:1 | O:0 | Y:1 | G:0 | B:0 | I:2 | V:0 ] // 0

Top Deck: GIV (92 cards remaining in deck)

Player 2, what would you like to do?
[1] Try to Score
[2] Try to Steal

>>

```

*Sample interface after Try to Score Flow B (notice **Player 1's B**)*

Try to Steal Sample Flow

```

P1 => [ R:1 | O:1 | Y:1 | G:1 | B:0 | I:0 | V:0 ] // 0
P2 => [ R:0 | O:1 | Y:0 | G:0 | B:2 | I:0 | V:1 ] // 0
P3 => [ R:1 | O:0 | Y:1 | G:0 | B:0 | I:2 | V:0 ] // 0

Top Deck: RGB (93 cards remaining in deck)

Player 1, what would you like to do?
[1] Try to Score
[2] Try to Steal

>> 2

Who would you like to steal from?
[1] Player 2
[2] Player 3

>> 2

```

Sample interface for the start of Player 1's turn

```

Resolving turn for Player 1...
- Drawn card color reveal: B (1 pt/s)!
- Player 2 has (2) B card/s!
- +3 (B) cards to Player 1's Tank!

```

```

Resolving turn Player 1...
- Drawn card color reveal: R (2 pt/s)!
- Player 2 has no R cards...
- Adding drawn card to Player 2's Tank

```

Sample interface for resolving the Try to Steal flow.

On the left (Try to Steal Flow A) is when the drawn card color exists in the other player's Tank.
On the right (Try to Steal Flow B) is when the drawn card color does not yet exist in the other player's Tank.

```

P1 => [ R:1 | O:1 | Y:1 | G:1 | B:3 | I:0 | V:0 ] // 0
P2 => [ R:0 | O:1 | Y:0 | G:0 | B:0 | I:0 | V:1 ] // 0
P3 => [ R:1 | O:0 | Y:1 | G:0 | B:0 | I:2 | V:0 ] // 0

Top Deck: GIV (92 cards remaining in deck)

Player 2, what would you like to do?

```

*Sample interface after Try to Steal Flow A (notice **Player 1** and **Player 2's B**)*

```

P1 => [ R:1 | O:1 | Y:1 | G:1 | B:0 | I:0 | V:0 ] // 0
P2 => [ R:1 | O:1 | Y:0 | G:0 | B:2 | I:0 | V:1 ] // 0
P3 => [ R:1 | O:0 | Y:1 | G:0 | B:0 | I:2 | V:0 ] // 0

Top Deck: GIV (92 cards remaining in deck)

Player 2, what would you like to do?

```

*Sample interface after Try to Steal Flow B (notice **Player 2's R**)*

Note on Taking Cards from the Deck: When drawing cards from the deck, always take the first card in the array (i.e., the card at index 0).

1.3.2 View Statistics

When the user is in this part of the program, the interface should display the Top 10 Players based on either (a) number of wins, or (b) highest score. For the project's base requirements, tracking each player's highest score can be simplified by overwriting it. However, you may choose a different method for high score tracking (see [Section 1.6](#) on bonus points).

1.3.3 Settings

When the user is in this part of the program, it must provide **at least** the following settings:

- **Set Winning Points:** Set the minimum number of points required to win the game (**default: 20 points**).
- **Set Shuffle Seed:** Set the seed value used for shuffling (**default: random**).

1.4 Interface

This project is text-based and should be run from the terminal or command line. Students have the flexibility to choose the type of user interface they wish to implement, provided it is intuitive and adheres to the specifications outlined in this document. Creativity in presenting the game is encouraged, as long as the specifications are followed.

There must be error checking for all player inputs. If the input comes from a list of options, then only the options should be allowed. If the input requires a string, then the input string must be valid. If the user enters an invalid input, the program should not crash. The program must display an error message and continue to prompt the user until an appropriate input is given.

1.4.1 Custom User Interface Library (Optional)

Use of this library is **optional**, but it can be useful for creative and adventurous students who want to deliver more visually appealing projects. The following files are for those who want to implement more advanced user interface features.

- **interface.c:** a C file containing some helper functions for advanced user interface operations
- **interfacesample.c:** a source file containing an example usage of the different features of the **interface.c** library, which can also be used to test if the features are working properly on your machine

The **interface.c** file contains the following useful functions that can help you design the interface of your game:

Function	Description
void iMoveCursor(int x, int y)	Moves the cursor to the given location on the command line. x and y are the coordinates of the desired location. You can use this before calling printf or scanf to change where you display or scan characters on the screen. Note that any existing text that is already displayed on that section of the screen may be overwritten.
void iSetColor(int color)	This function sets the color of succeeding output statements, like printf . The color parameter is an integer representing the desired color (see color list below). All succeeding display will be of the specified color until this function is called again with a different parameter.
void iClear(int x, int y, int w, int h)	Clears a rectangular portion of the screen, starting from the position at (x , y) and having a width and height of w and h , respectively. For example, the call iClear(0,0,5,5) deletes all text in a 5 × 5 rectangle from the top-left corner of the terminal. Note that this function also moves the cursor to (x , y) after the clearing operation.
void iHideCursor()	Hides the text cursor (visually).
void iShowCursor()	Shows the text cursor (visually).

For the `iSetColor()` function, the parameter is one of these color codes. You may pass either the variable names (which hold integer constants) or the actual integer value corresponding to the color that you want. Passing any other value aside from these may cause unpredictable results. For this project, you are limited to these colors only.

To use the functions in `interface.c` in your own program, you must first reference it using `#include "interface.c"`. Please refer to the `interfacesample.c` source file to see an example of how to use the custom library. This source file may also be compiled and run to check if the library is working properly. Try to compile and run this program, and you should see a short working program demonstrating the various interface functionalities.

Color	Variable	Integer Code
White / reset	I_COLOR_WHITE	0
Red	I_COLOR_RED	1
Blue	I_COLOR_BLUE	2
Green	I_COLOR_GREEN	3
Yellow	I_COLOR_YELLOW	4
Cyan	I_COLOR_CYAN	5
Purple	I_COLOR_PURPLE	6

1.5 Mantis Card Format

There are 7 colors in this game. To make things simple, we'll use the colors of the rainbow: red, orange, yellow, green, blue, indigo, and violet (ROYGBIV). The "front" and "back" of the Mantis cards have an important connection:

- The front (face) of each card has a single color (e.g., R).
- The back of each card shows 3 different colors (e.g., RGB).
- The color on the front is always 1 of the 3 colors from the back of the card.

Each color card (based on the front) has 12 cards, making a total of 84 cards in a standard Mantis game deck. A modification in this text-based version of the game is that each card now has an associated point value of 1, 2, or 3.

Each card is encoded in the `mantis.txt` in the following format:

```
<front color> | <back color 1><back color 2><back color 3> <point value><newline>
```

Sample contents of `mantis.txt`:

```
R | RGB 2<newline>
Y | OYI 3<newline>
V | BIV 1<newline>
...
O | ROY 1<newline>
G | YGI 1<newline>
<end of file>
```

If you opt to use colors, here is the suggested mapping:

```
R ----- (red)
O ----- (white)
Y ----- (yellow)
G ----- (green)
B ----- (blue)
I ----- (cyan)
V ----- (purple)
```

1.6 Bonus Points

Students can earn **up to 10 bonus points (exceeding the maximum score)** by implementing additional features that will enrich the quality of the project. Bonus points will only be given for meaningful and non-trivial additions to the base project (i.e., simply using different font colors is considered trivial, and thus will not incur bonus points). Ideas for bonus features include, but are not limited to:

- More interesting game statistics
 - Based on the current project specifications, the only other player data stored are the number of games won and their highest scores. You can explore storing more player data and presenting more interesting statistics.
- Game manager

- This feature is a facility where users can manage the various game components, such as player management.
- Two-player variant
 - Mantis has variant rules when playing with only 2 players, such as first to 15 cards in the Score Pile instead of 10, and extra turns. You may refer to the instruction manual for more details on this variant.
- Save and continue game
 - An ongoing unfinished game may be saved for the meantime and continued sometime later.
- Good user experience (UX) design

Bonus points will only be given to projects that have met all the minimum requirements. The amount of bonus points is up to the discretion of the instructor. Ensure that additional features do not conflict with the minimum requirements stated in the specifications; otherwise, these may result in some of those requirements not being met. When in doubt, please consult your instructor.

1.7 Implementation Requirements

The project must abide by the following implementation guidelines:

- The program should be **fully implemented in the C programming language** (C99, NOT C++).
- Observe the appropriate use of conditional statements, loops, and functions.
 - **Do not use brute force.**
 - Codes must be modular (i.e., split into several logical functions). Codes that are not modularized properly will not be accepted, even if the program works properly.
- You can use these in the future when you have much more experience and wisdom in programming. However, **while you are in the PROG series, you are NOT ALLOWED to do the following:**
 - ✗ to declare and use global variables (i.e., variables declared outside any function)
 - ✗ to use **exit**, **goto** (i.e., to jump from code segments to code segments), **break** (except in **switch** statements), or **continue** statements
 - ✗ to use **return** statements to prematurely terminate a loop, function, or program
 - ✗ to use **return** statements in void functions
 - ✗ to call the **main()** function to repeat the process instead of using loops
- You may only use the standard C libraries.
- You may use topics outside the scope of CCPROG2, but this will be **self-study**.
- Adhere to coding conventions and **must include proper documentation** as indicated in [Section 2.1](#).
- Debug and test your program exhaustively. The submitted program should have
 - **NO syntax errors and warnings**
 - It should compile properly with the command:


```
gcc -Wall -std=c99 <yourMP.c> -o <output>
```
 - **NO logical errors**
 - Based on the test plan that the program was subjected to
 - As part of the requirements, you are required to create a test plan and execute your program against it. See [Section 2.2](#) for more details.

2 Documentation

2.1 Internal Code Documentation

You are **required** to have the following internal documentation in your source code.

2.1.1 Program Structure

The structure of your program should look like this:

```
/**
```

```

* Description      : <short description of the project>
* Author/s         : <student1 full name (last name, first name)>
*                  : <student2 full name (last name, first name)>
* Section          : <your section>
* Last Modified    : <date when last revision was made>
* Acknowledgments : <list of references used in the making of this project>
*/

/* preprocessor directives */

/* definitions (i.e., constants, typedefs, structs) */

/* function implementations */

int main()
{
    /* your project code */

    return 0;
}

/**
 * This is to certify that this project is my/our own work, based on my/our personal
 * efforts in studying and applying the concepts learned. I/We have constructed the
 * functions and their respective algorithms and corresponding code by myself/ourselves.
 * The program was run, tested, and debugged by my/our own efforts. I/We further certify
 * that I/we have not copied in part or whole or otherwise plagiarized the work of other
 * students and/or persons, nor did I employ the use of AI in any part of the deliverable.
 *
 * <student1 full name (last name, first name)> (DLSU ID# <number>)
 * <student2 full name (last name, first name)> (DLSU ID# <number>)
 */

```

A template of the program structure is provided for you. See [Section 3.4](#) on how to access it.

2.1.2 Functions and Function Comments

For **each function you define**, add the following comments. These serve as documentation to describe what the function does, what each parameter is for, and what is being returned, if any. If applicable, include pre-conditions as well. Pre-conditions refer to the assumed state of the parameters. [Refer to this link for more information about C code documentation](#). Follow the format below when writing functions and function comments:

```

/**
 * <Description of what the function does>
 * @param <name1> <description of parameter 1>
 * @param <name2> <description of parameter 2>
 * ...
 * @param <nameN> <description of parameter N>
 * @return <description of the return value>
 * @pre <list of pre-conditions>
 */
<return type> <function name>(<parameter list>)
{ // open brace is at the beginning of the new line, aligned with matching close brace
    <variable declarations> // variable declarations should be before the start of any
                          // statements, not inserted in the middle of the loop,
                          // to promote readability
    <rest of the function implementation>
} // close brace is aligned to the matching open brace

```

Below is a sample function with a function comment adhering to the format indicated above:

```
/**
 * Computes the average of the non-negative numbers from a given list of numbers
 * @param arr The starting address of the array containing the list of numbers
 * @param arrSize The size of the array
 * @return The average of the non-negative numbers from the list of numbers
 * @pre arr can include positive numbers, negative numbers, and zeros
 * @pre arrSize is the correct size of arr
 */
float getAverage(int arr[], int arrSize)
{
    float average;
    int sum = 0, n = 0, i;

    for (i = 0; i < arrSize; i++)
        if (arr[i] >= 0)
        {
            sum += arr[i];
            n++;
        }

    if (n == 0) // prevent divide by zero
        average = 0;
    else
        average = sum / (float);

    return average;
}
```

2.1.3 Structures

For **each structure you define**, add the following comments. These serve as documentation to describe the structure and its members. On the right is a sample structure with documentation adhering to the format indicated on the left:

```
/**
 * <Description of the structure>
 */
struct struct_name
{
    <data type> <member1>; // Description of member1
    <data type> <member2>; // Description of member2
    ...
    <data type> <memberN>; // Description of memberN
};
```

```
/**
 * Represents a 2D point
 */
struct coordinate
{
    int x; // The x-coordinate of a point
    int y; // The y-coordinate of a point
};
```

2.2 Test Plan

As part of the requirements of this project, you are also required to submit a test plan document. This document shows the tests done to ensure the correctness of the program.

- It should be presented in a table format. Refer to [Appendix A](#) for an example.
 - A template of the test plan is provided for you. See [Section 3.4](#) on how to access it.
- There should be **at least three test cases per function** (as indicated in the Test Description).
 - There is no need to test functions that are intended for interface display/design.
- Test descriptions are supposed to be unique and should indicate classes/groups of test cases on what is being tested.

3 Deliverables

3.1 Submission & Demo

- **The final MP deadline is March 30, 2026 (M) 0730 via AnimoSpace.** After the indicated time, the submission page will be locked and thus considered no submission (equivalent to 0 in the MP grade).
- The schedule for the demo will be announced at a later date.
- During the demo, you are expected to **appear on time**, be able to **answer questions about the implementation of the project**, and/or be able to **revise the program based on a given demo problem**. Note that more than just delivering a working program, the most important outcome of this project is for you to learn and have a good understanding of how your program works. Therefore, failure to meet any of these expectations could result in a grade of 0.
- During the MP demo, it is expected that the program can be compiled successfully and will run. **If the program does not run, the grade for the project is automatically 0.** However, a running program with complete features may not necessarily get full credit, as implementation (i.e., code) and other submissions (e.g., non-violation of restrictions evident in code, test script, and internal documentation) will still be checked.

3.2 Deliverables Checklist

The following are the deliverables:

- ☐ Upload via AnimoSpace submission a zip file (see right on how the zip file content is organized) containing the following:
 - ☐ Source code
 - ☐ Test plan PDF
 - ☐ Text files
- ☐ Email this zip file as an attachment to YOUR own DLSU email address on or before the deadline (please do not CC the instructor)

Your submission should have this folder structure before compressing it into a zip file:

```
src/
|-- mp.c
|-- mantis.txt
|-- players.txt
|-- TestPlan.pdf
|-- <any other files and folders used as
part of this project>
```

3.3 Milestones

There is only 1 deadline for this project: **March 30, 2026 (M) 0730**, but the following are suggested milestones:

MS-0: Design and Planning (Feb 7, 2026)

- ☐ Read through the project specifications

MS-1: Program Structure (Feb 14, 2026)

- ☐ Menu options and transitions
- ☐ Preliminary outline of the functions to be created

MS-2: Game Proper, Part 1 (Feb 28, 2026)

- ☐ Declaration/implementation of data structures
- ☐ Game Proper > Player selection (dummy data)
- ☐ Game Proper > Shuffle cards and deal Tank (dummy data)
- ☐ Game Proper > Try to Score Flow

MS-3: Game Proper, Part 2 (Mar 7, 2026)

- ☐ Game Proper > Try to Steal Flow
- ☐ Game Proper > Show game results

MS-4: File Handling (Mar 14, 2026)

- ☐ Load players from a **players.txt**
- ☐ Save a new player to **players.txt**
- ☐ Load mantis cards from **mantis.txt**
- ☐ Optional: Implement bonus features

MS-5: View Statistics (Mar 21, 2026)

- ☐ View Statistics > Top Players
- ☐ Optional: Implement bonus features

MS-6: Finalization (Mar 28, 2026)

- ☐ Integration testing (test program as a whole, not per function)
- ☐ Double-check that all base requirements of the project are met
- ☐ Optional: Implement bonus features
- ☐ Zip and submit the final MP-related files

Note that each milestone assumes fully debugged and tested code/function, code written following coding convention, and includes internal documentation and documented tests in the test plan. It is expected that at least one submission per target milestone is done in AnimoSpace. Even if this is not graded, this serves as version control and also proof of continuing efforts.

3.4 MP Starter Files

- The [MP starter files](#) contain the following:
 - A template of the MP program structure
 - **defs.h, helpers_1.c, helpers_2.c, mp_template.c**
 - The custom user interface library
 - **interface.c, interfacesample.c**
 - The list of unshuffled mantis cards (**mantis.txt**)
- A [template of the test plan can be accessed at this link](#)
 - Refer to [Appendix B](#) for the guide on how to export the template as a PDF.

4 Collaboration and Academic Honesty

A student cannot discuss or ask about design or implementation with other persons, with the exception of the teacher and their groupmate. Copying other people's work and/or working in collaboration with other teams are not allowed and are punishable by a grade of 0.0 for the entire CCPROG2 course, and a case may be filed with the Discipline Office. Comply with the policies on collaboration and AI usage as discussed in the course syllabus.

5 Grading

For grading, refer to the MP rubrics indicated in the syllabus

Acknowledgments / Disclosure

The rules of the [Mantis game by Exploding Kittens](#) were adapted for this text-based game project.

Additionally, the following Generative AI tools were used to assist in the making of this document:

- [ChatGPT](#) was used mainly for streamlining and/or sentence construction. The instructor validated the AI-generated output and modified it as needed.

Appendix A: Test Plan Format Sample

Function	#	Test Description	Test Input	Expected Output	Actual Output	P/F
getAverage	1	arr contains only non-negative numbers	arr = {1,2,3,4,5,0} arrSize = 6	2.5		
	2	arr contains only negative numbers	arr = {-1,-2,-3,-4,-5} arrSize = 5	0.0		
	3	arr contains a mix of positive numbers, negative numbers, and zeros	arr = {-1,0,5,-4,7,1} arrSize = 6	3.25		
	4	arr contains only zeros	arr = {0, 0, 0} arrSize = 3	0.0		
anotherFunction	1	...	...	...		
	2	...	...	...		
...	...	...	...	...		

Given the sample function in [Section 2.1.2](#), the following are 4 distinct classes of tests:

1. Testing with arr containing only non-negative numbers
 - Non-negative numbers (positive numbers and zeros) are the only values that can be part of the average computation
2. Testing with arr containing only negative numbers
3. Testing with arr containing a mix of positive numbers, negative numbers, and zeros.
4. Testing with arr containing only zeros
 - Zero is a non-negative number

The following test descriptions are **incorrectly** formed:

- **Too specific:** Testing with arr = {1, 2, 3, 4, 5}
- **Too general:** Testing if the function can correctly compute the average of the non-negative numbers
- **Not necessary (because already defined in the pre-condition):** Testing with a negative arrSize

A [template of the test plan can be accessed at this link](#). Refer to [Appendix B](#) for the guide on how to export the template as a PDF.

Appendix B: Export Test Plan Guide

1. In the **TestPlan** sheet:

File > Download > PDF (.pdf)

2. Adjust the settings to:

- Export: Current Sheet
- Paper size: Legal (8.5 x 14)
- Page orientation: Landscape
- Scale: Fit to width
- Margins: Narrow
- Formatting dropdown
 - Leave as is
- Headers& footers dropdown
 - Check: Current date
 - Check: Current time
 - Uncheck: Repeat frozen rows

3. Click Export.

CANCEL **EXPORT**

Export

Current sheet

Paper size

Legal (8.5" x 14")

Page orientation
☒ Landscape ☐ Portrait

Scale

Fit to width

Margins

Narrow

[SET CUSTOM PAGE BREAKS](#)

Formatting ▾

Headers & footers ▴

☐ Page numbers

☐ Workbook title

☐ Sheet name

☒ Current date

☒ Current time

[EDIT CUSTOM FIELDS](#)

Row & column headers
Go to View > Freeze to select which rows/columns to repeat on all pages

☐ Repeat frozen rows

☐ Repeat frozen columns